

LANGGRAPH E ORQUESTRAÇÃO BASEADA EM GRAFOS

Cecilia Rufatto Felipe Proença Leonardo Moino Leonardo Quartaroli
Lorena Scabello
Agentes de IA | Grupo G1

1. Contexto e motivação

Frameworks lineares como o LangChain tradicional executam etapas em ordem fixa. Funciona bem em fluxos curtos e determinísticos, mas agentes de verdade costumam precisar de coisas que esse modelo não entrega de forma nativa:

- ciclos, pra refletir ou retentar uma etapa;
- estado persistente entre execuções;
- decisões condicionais que dependem do que aconteceu antes;
- pausa e retomada com intervenção humana.

O LangGraph cobre essa lacuna tratando o agente como um grafo: cada nó é uma função sobre o estado, as arestas decidem o caminho, e *checkpointers* oficiais cuidam da persistência. É a base do LangSmith, do LangChain Academy e de agentes em produção em empresas como Klarna, Replit e Elastic.

2. Conceitos-chave

StateGraph	Grafo cujo estado é um <code>TypedDict</code> compartilhado entre nós.
Node	Função Python que recebe e devolve estado.
Edge	Conexão direta entre dois nós (<code>add_edge</code>).
Conditional Edge	Roteamento dinâmico via função <code>router(state)</code> .
Reducer	Estratégia de <i>merge</i> de campos do estado (ex. <code>add_messages</code>).
Checkpoint	Persistência do estado entre execuções (SQLite, Postgres, Redis).
HITL	<i>Human-in-the-loop</i> : pausa com <code>interrupt</code> e retoma via <code>Command</code> .

3. Estado da arte

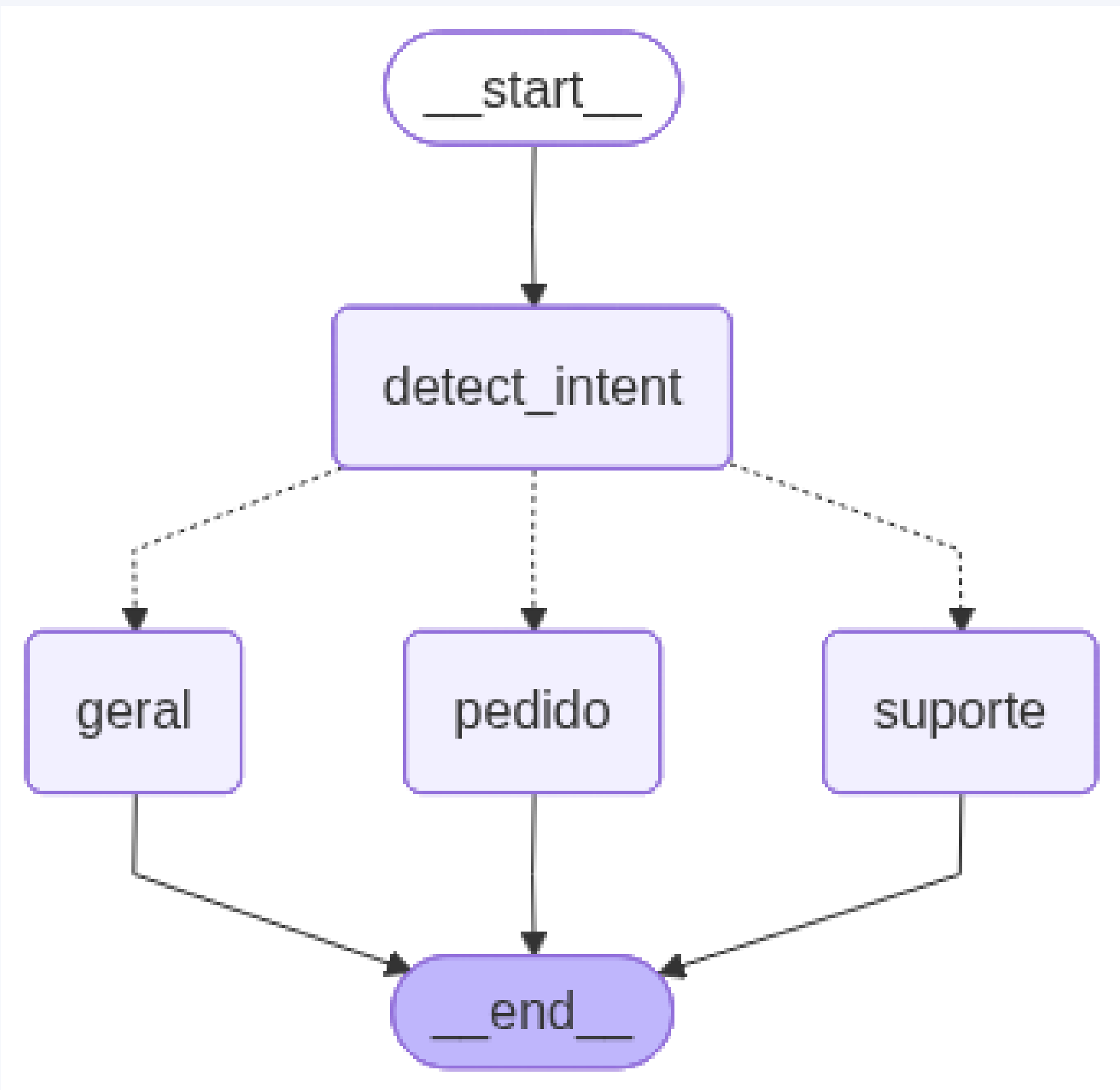
Linha do tempo

2022	LangChain — <i>chains</i> lineares para composição de LLMs.
Jan 2024	LangGraph 0.0.x — primeira versão pública.
Mai 2024	LangGraph Studio + Cloud — observabilidade e deploy.
2025	LangGraph 1.x estável — <i>checkpointers</i> , durable execution.

Comparativo com frameworks concorrentes

Framework	Estado	Ciclos	Persistência
LangGraph	Tipado	Nativo	Checkpointers oficiais
CrewAI	Implícito	Limitado	Memory module
AutoGen	Mensagens	Por chat	Externa
OpenAI Swarm	Handoffs	Não	Não (experimental)

4. Arquitetura do protótipo



Cada mensagem que entra é processada pelo nó `detect_intent`, que rotula como *pedido*, *suporte* ou *geral*. A aresta condicional `route_intent` despacha pro handler correspondente, que responde e grava o histórico em JSON pelo `memory.py` antes de chegar ao `END`.

O estado é um `TypedDict` (`AgentState`) com os